

Managing Clusters

Kubernetes Production · Lesson 11

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Use **Namespaces** to isolate resources and switch context
- Create and consume a **ConfigMap** (env, envFrom, volume, subPath)
- Create and consume a **Secret**, and explain why base64 ≠ encryption
- Constrain a namespace with a **ResourceQuota**
- Inject defaults and per-container bounds with a **LimitRange**

All five objects are `apiVersion: v1` — kubernetes.io/docs/concepts

11.1 Namespaces

What Is a Namespace?

A **Namespace** isolates groups of resources within a **single cluster**.

- Resource **names must be unique within a namespace**, not across them
- Applies to **namespace-scoped** objects (Deployments, Services, Pods...)
- Cluster-wide objects (Nodes, PersistentVolumes, StorageClass) are **not** namespaced
- Use them for **many users across teams**; for a few users, prefer **labels**

Production tip: consider **not** using the `default` namespace.

The Four Initial Namespaces

Namespace	Purpose
default	Use the cluster without first creating a namespace
kube-system	Objects created by the Kubernetes system
kube-public	Readable by all clients; public cluster-wide data
kube-node-lease	Holds Lease objects for node heartbeats

- Avoid the kube- prefix — reserved for system namespaces
- Control plane sets an immutable label `kubernetes.io/metadata.name`

Namespaced vs Cluster-Scoped

```
# List namespaces
kubectl get namespace

# Create a namespace
kubectl create namespace dev

# Which resources live IN a namespace?
kubectl api-resources --namespaced=true

# Which resources are cluster-wide?
kubectl api-resources --namespaced=false
```

CKA tip: `--namespaced=false` quickly tells you whether an object (e.g. a Node or PV) can be placed in a namespace.

Switching Namespace Context

```
# One-off: target a namespace for a single command
kubectl get pods --namespace=dev
kubectl run nginx --image=nginx -n dev

# Persistent: change the default namespace for your context
kubectl config set-context --current --namespace=dev

# Verify the active namespace
kubectl config view --minify | grep namespace:
```

- Service DNS: `<service-name>.<namespace-name>.svc.cluster.local`
- Same namespace → short `<service-name>`; cross-namespace → FQDN

11.2 ConfigMaps

What Is a ConfigMap?

A **ConfigMap** (`apiVersion: v1`) stores **non-confidential** data as key-value pairs, decoupling config from the container image.

- Consumed as **env vars**, **command args**, or **files in a volume**
- **Not** secret and **not** encrypted — for confidential data use a Secret
- Size limit **1 MiB**; `data` = UTF-8 strings, `binaryData` = base64
- `immutable: true` (since v1.19) prevents updates and improves performance

ConfigMap and the Pod that uses it must be in the **same namespace**.

Creating a ConfigMap

```
# From literal key-value pairs
kubectl create configmap game-demo \
  --from-literal=player_initial_lives=3

# From a file (key = filename) or a whole directory
kubectl create configmap app-config --from-file=game.properties
kubectl create configmap app-config --from-file=config-dir/
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  player_initial_lives: "3"
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
```

Consume as Env Vars

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo-pod
spec:
  containers:
    - name: demo
      image: alpine
      env:
        - name: PLAYER_INITIAL_LIVES      # single key
          valueFrom:
            configMapKeyRef:
              name: game-demo
              key: player_initial_lives
      envFrom:
        - configMapRef:                  # all keys at once
            name: game-demo
```

Env vars are **not** auto-updated when the ConfigMap changes — restart the Pod.

Consume as a Mounted Volume

```
spec:
  containers:
    - name: demo
      image: redis
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: game-demo          # each key becomes a file under /etc/config
```

- Omit `items` → every key is mounted as a file named after the key
- **Volume-mounted** ConfigMaps are eventually updated automatically

Mounting a Single Key with subPath

```
volumeMounts:
- name: config
  mountPath: /etc/config/game.properties
  subPath: game.properties          # mount ONE key as a single file
  readOnly: true
volumes:
- name: config
  configMap:
    name: game-demo
```

- `subPath` injects one key **without hiding** other files in the directory
- Trade-off: a `subPath` mount does **not** receive automatic updates

11.3 Secrets

What Is a Secret?

A **Secret** (`apiVersion: v1`) stores a **small amount of sensitive data** — passwords, tokens, keys — out of Pod specs and images.

- `data` holds **base64-encoded** values; `stringData` takes plaintext (auto-encoded)
- Size limit **1 MiB**; supports `immutable: true`
- **base64 is encoding, NOT encryption** — anyone with API/etcd access can read it

Hardening: enable **Encryption at Rest** (`EncryptionConfiguration`), apply least-privilege **RBAC**, and restrict Secret access to specific containers.

Secret Types

Type	Usage
Opaque	Arbitrary user-defined data (default)
kubernetes.io/service-account-token	ServiceAccount token
kubernetes.io/dockerconfigjson	Private registry pull credentials
kubernetes.io/basic-auth	Basic-auth username/password
kubernetes.io/ssh-auth	SSH credentials
kubernetes.io/tls	TLS cert + key (tls.crt , tls.key)
bootstrap.kubernetes.io/token	Bootstrap token data

Creating a Secret

```
# Generic secret from literals
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=s3cr3t

# TLS secret from cert + key files
kubectl create secret tls tls-secret \
  --cert=path/to/cert.crt --key=path/to/key.key
```

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
stringData:      # plaintext, encoded for you on create
  username: admin
  password: s3cr3t
```

Consuming a Secret

```
spec:
  containers:
  - name: app
    image: redis
    env:
      - name: PASSWORD
        valueFrom:
          secretKeyRef:      # one key as an env var
            name: mysecret
            key: password
    envFrom:
      - secretRef:          # all keys as env vars
        name: mysecret
    volumeMounts:
      - name: secret-volume
        mountPath: /etc/secrets
        readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: mysecret  # keys become files under /etc/secrets
```

Inspecting a Secret

```
# View the (base64-encoded) Secret
kubectl get secret mysecret -o yaml

# Decode a single value
kubectl get secret mysecret \
  -o jsonpath='{.data.username}' | base64 -d
```

- Mounted Secret files appear at `/etc/secrets/<key>`
- Volume-mounted Secrets refresh automatically; env vars require a Pod restart

CKA tip: `base64 -d` to decode — the exam often hands you a Secret and asks for the plaintext value.

11.4 ResourceQuota

What Is a ResourceQuota?

A **ResourceQuota** (`apiVersion: v1`) limits **aggregate** resource consumption **per namespace** so one team can't monopolize the cluster.

- A cluster admin creates **at least one per namespace**
- The control plane **rejects requests with HTTP 403** when a quota is exceeded
- Constrains **compute** (cpu/memory), **storage**, and **object counts**

When a compute quota is set, **every new Pod must declare requests/limits** for that resource — otherwise its creation is rejected.

Quota Dimensions

Category	Example fields
Compute	requests.cpu , requests.memory , limits.cpu , limits.memory
Storage	requests.storage , persistentvolumeclaims
Object count	pods , services , configmaps , secrets , count/<resource>

```
kubectl create quota my-quota \
  --hard=requests.cpu=1,requests.memory=1Gi,limits.cpu=2,limits.memory=2Gi

kubectl describe quota my-quota -n myteam      # shows Used vs Hard
```

ResourceQuota Manifest

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: myteam
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    persistentvolumeclaims: "10"
    pods: "100"
```

- `kubectl describe quota` shows a **Used vs Hard** table per resource
- The 101st Pod (or one exceeding `requests.cpu`) is **rejected with 403**

11.5 LimitRange

What Is a LimitRange?

A **LimitRange** (`apiVersion: v1`) constrains resource allocations for **individual** Pods, Containers, and PVCs in a namespace.

- Sets **min / max** compute per Pod/Container (and storage per PVC)
- **Auto-injects** `default` (limit) and `defaultRequest` when a container omits them
- Enforces `maxLimitRequestRatio` between limit and request
- Applied by the **LimitRange admission controller** at Pod admission time

Complementary to ResourceQuota (namespace totals) — use them **together**.

LimitRange Manifest

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-mem-constraints
  namespace: myteam
spec:
  limits:
  - type: Container
    default:                # default LIMIT if unset
      cpu: 500m
      memory: 512Mi
    defaultRequest:        # default REQUEST if unset
      cpu: 100m
      memory: 128Mi
    min:                   # smallest allowed per container
      cpu: 100m
    max:                   # largest allowed per container
      cpu: "1"
```

How LimitRange + ResourceQuota Combine

1. A user creates a Pod with **no** `resources` set
2. **LimitRange** injects `requests` / `limits` from its defaults at admission
3. **ResourceQuota** sums the injected requests against the namespace **hard** total
4. If the namespace would go **over quota** → request **rejected with 403**
5. If a container's request falls outside LimitRange `min` / `max` → also **403**

CKA tip: pairing them lets bare Pods get sane defaults *and* still be counted against the namespace quota — neither works as well alone.

Key Takeaways

- **Namespaces** isolate namespaced objects; switch with `kubect1 config set-context --current --namespace=`
- **ConfigMap** = non-secret config; consume via `env`, `envFrom`, volume, `subPath`
- **Secret** = base64 (NOT encryption); enable **encryption at rest** + RBAC
- **ResourceQuota** caps a namespace's totals; over-quota requests get **403**
- **LimitRange** sets per-container defaults/bounds and **auto-injects** requests/limits

End of Lesson 11

Next: Lesson 12 — Scheduling

```
kubectl create ns · kubectl create configmap · kubectl create secret generic · kubectl describe quota
```